Concept 1: What is SQL?

SQL, which stands for **Structured Query Language**, is the standard programming language used to communicate and interact with relational databases.

Think of a relational database as a collection of highly organized, linked spreadsheets (tables). SQL is the specific language you use to ask questions about that data or to reorganize it.

Here are the primary functions of SQL:

- It is utilized to store, retrieve, update, and delete the data held within the database.
- It is also used to define, create, and modify the actual database structures themselves, such as the tables, views, and indexes.

Query Example

SELECT * FROM Employees;

Explanation: This is one of the most fundamental SQL queries. It retrieves all of the records (every row and every column) from the table named `Employees`. The `*` acts as a wildcard meaning "everything."

Concept 2: What is a Database?

A database is an organized collection of data that is electronically stored and managed. Its primary purpose is to allow users to efficiently store, retrieve, update, and manage data.

Databases are critical infrastructure for handling large volumes of information across various platforms, including websites, business systems, and software applications.

Real-World Example

In an e-commerce application, a customer database typically stores specific details such as the customer's name, email address, contact number, and their complete purchase history.

Additional Context

While a "database" can technically be any collection of data, in the context of SQL, it specifically refers to a **Relational Database Management System (RDBMS)**. In an RDBMS, this organized collection of data is strictly structured into tables consisting of rows (records) and columns (attributes), allowing you to establish logical relationships between different sets of data.

Concept 3: Types of SQL Commands

SQL commands are divided into five specific categories based on what they actually do within the database.

- **DDL (Data Definition Language):** This defines the structural blueprint of your database. Commands include CREATE (make a new table), ALTER (change an existing table), DROP (delete a table entirely), and TRUNCATE (empty a table).
- +1
- **DML (Data Manipulation Language):** This is used to manage and modify the actual data living inside your tables. Commands include INSERT (add data), UPDATE (change data), and DELETE (remove data). The PDF also lists SELECT here, though it is primarily DQL.
- +1
- **DCL (Data Control Language):** This manages security and permissions. Commands include GRANT (give a user permission) and REVOKE (take permission away).
- +1
- **TCL (Transaction Control Language):** This manages the transactions (batches of DML operations) in the database to ensure data integrity. Commands include COMMIT (save changes permanently), ROLLBACK (undo changes), and SAVEPOINT (set a rollback point).
- +1
- **DQL (Data Query Language):** This is solely dedicated to fetching and retrieving data from the database. The primary command is SELECT.

Concept 4: What is a Primary Key?

A Primary Key is a specific column (or a combination of a few columns) that acts as a unique identifier for every single row in a table. Think of it like a national ID number or a fingerprint for each record.

- It absolutely cannot contain NULL (empty) values.
- It cannot have duplicate values; every entry must be 100% unique.
- You are only allowed to have one primary key per table.

The Query

```SQL
CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   Name VARCHAR(50),
   Age INT
);
```
``` [cite: 50, 51, 52, 53, 54]

**Input Data (To Insert)**
Let's say we run the following insert commands:
`INSERT INTO Employees (EmployeeID, Name, Age) VALUES (1, 'Alice', 28);`
`INSERT INTO Employees (EmployeeID, Name, Age) VALUES (2, 'Bob', 34);`
`INSERT INTO Employees (EmployeeID, Name, Age) VALUES (1, 'Charlie', 25);` -- *This will fail!*
```

Output Table
Because `EmployeeID` is the Primary Key, the database will reject "Charlie" because the ID of "1" is already taken by Alice.

EmployeeID	Name	Age
1	Alice	28
2	Bob	34

Concept 5: What is a Foreign Key?

A Foreign Key is a column in one table that directly references the Primary Key of a completely different table[cite: 56, 57]. This is how relational databases link data together. It establishes a strict relationship between tables to enforce "referential integrity" (meaning you cannot link to a record that doesn't exist)[cite: 58].

* Unlike Primary Keys, Foreign Keys *can* contain duplicate values (many employees can belong to one department)[cite: 61].
* They can also accept `NULL` values (an employee might not be assigned a department yet)[cite: 62].

The Query
```sql
CREATE TABLE Departments (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(50)
);

CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   Name VARCHAR(50),
   DepartmentID INT,
   FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```
[cite: 65, 66, 67, 68, 69, 71, 72, 73, 74]

Input Tables (Before Query Execution)

Departments Table
DepartmentID (Primary Key)	DepartmentName
100	Human Resources
101	Engineering

Output / Resulting Employee Table
If we insert employees and link them to the departments above, it looks like this. If we try to insert an employee with a `DepartmentID` of "999", the database will block it because "999" does not exist in the Departments table.

EmployeeID	Name	DepartmentID (Foreign Key)
1	Alice	101
2	Bob	100
3	Charlie	101
4	Diana	NULL

Concept 6: What is a UNIQUE Key?

A UNIQUE Key is a constraint that ensures all values in a specific column (or combination of columns) are distinct across all rows in that table. It prevents you from entering duplicate values into that column, but it generally allows `NULL` values. In most databases, only one `NULL` value is permitted per unique column.

+2

- It ensures the uniqueness of each record in the column.
- Unlike a Primary Key, multiple UNIQUE keys can be defined within a single table.
- It is a key tool for maintaining data integrity.

The Query

```SQL
CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   Email VARCHAR(100) UNIQUE,
   Name VARCHAR(50)
);
```

Input Data (To Insert)

Let's insert some data. The third insert will fail.

```
INSERT INTO Employees (EmployeeID, Email, Name) VALUES (1, 'alice@email.com', 'Alice');

INSERT INTO Employees (EmployeeID, Email, Name) VALUES (2, 'bob@email.com', 'Bob');

INSERT INTO Employees (EmployeeID, Email, Name) VALUES (3, 'alice@email.com', 'Charlie'); -- Fails due to duplicate email!
```

`INSERT INTO Employees (EmployeeID, Email, Name) VALUES (4, NULL, 'Diana');` -- *Succeeds.*

Output Table

EmployeeID	Email	Name
1	alice@email.com	Alice
2	bob@email.com	Bob
4	NULL	Diana

Concept 7: What is the Difference Between Primary Key and UNIQUE Key?

While both enforce uniqueness, they serve slightly different purposes in database design.

Feature	Primary Key	UNIQUE Key
Purpose	Uniquely identifies each row in a table.	Ensures all values in the column are unique without being a primary identifier.
NULL Values	Does not allow `NULL` values.	Allows one `NULL` value (in most databases).
Quantity	Only one primary key is allowed per table.	Multiple UNIQUE keys can be defined in a table.
Indexing	Automatically creates a unique clustered index.	Creates a unique non-clustered index.

Concept 8: What is NOT NULL Constraint?

The NOT NULL constraint strictly dictates that a column cannot accept NULL (empty) values. It is used to enforce mandatory data entry; every row absolutely must have a value in that specific column. It can be applied to one or more columns in a table.

+3

The Query

SQL

```
CREATE TABLE Employees (
   EmployeeID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   Email VARCHAR(100)
);
```

Input Data (To Insert) In this setup, Name cannot be empty, but Email can be. INSERT INTO Employees (EmployeeID, Name, Email) VALUES (1, 'Alice', 'alice@email.com'); INSERT INTO Employees (EmployeeID, Name, Email) VALUES (2, 'Bob', NULL); -- *Succeeds.* INSERT INTO Employees (EmployeeID, Name, Email) VALUES (3, NULL, 'charlie@email.com'); -- *Fails due to NOT NULL constraint on Name!*

Output Table

EmployeeID	Name	Email
1	Alice	alice@email.com
2	Bob	NULL

Concept 9: What is Default Constraint?

The Default Constraint provides a pre-set, default value for a column if no value is explicitly provided by the user during the insertion of a new record.

- It automatically assigns the default value if no input is given.
- It is highly useful for avoiding NULL values in specific columns.
- It can be applied to any data type.

The Query

SQL

```sql
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10, 2) DEFAULT 5000
);
```

+1

Input Data (To Insert) Notice how we completely skip the `Salary` column in the second insert. `INSERT INTO Employees (EmployeeID, Name, Salary) VALUES (1, 'Alice', 8000); INSERT INTO Employees (EmployeeID, Name) VALUES (2, 'Bob');`

Output Table Bob is automatically assigned the default salary of 5000.

EmployeeID	Name	Salary
1	Alice	8000.00
2	Bob	5000.00

Concept 10: Difference Between DELETE, TRUNCATE, and DROP

These three commands are used to remove data, but they operate at entirely different levels of severity and speed.

Command	Function	Can Rollback?	Affects Structure?	Speed
DELETE	Removes specific rows based on a condition using the `WHERE` clause.	Yes (with COMMIT/ROLLBACK).	No.	Slow (Row-by-row deletion).

TRUNCATE	Removes all rows from the table without a condition.	No.	No.	Faster than DELETE.
DROP	Deletes the entire table including data and structure.	No.	Yes (Removes table structure).	Fastest.

The Queries

Assume we have a populated `Employees` table.

- **DELETE:** `DELETE FROM Employees WHERE EmployeeID = 101;` (Deletes specific row)
- **TRUNCATE:** `TRUNCATE TABLE Employees;` (Deletes all rows, leaves an empty table shell behind)
- **DROP:** `DROP TABLE Employees;` (Deletes the table completely; it no longer exists in the database)

Alright, let's dive deep into the next batch! We're getting into some of the most critical parts of SQL now: filtering aggregated data and combining tables. You can copy this straight into your notes document.

Concept 11: What is the Difference Between WHERE and HAVING?

Both `WHERE` and `HAVING` are used to filter records, but they operate at entirely different stages of the query execution process.

- **WHERE:** This is a **row-level filter**. It filters individual records *before* any grouping takes place. It is used with `SELECT`, `UPDATE`, and `DELETE`.
- +2
- **HAVING:** This is a **group-level filter**. It is exclusively used to filter groups of records *after* the `GROUP BY` clause has aggregated them. It is only used with `SELECT` statements containing a `GROUP BY`.
- +2

The Queries

```SQL
-- WHERE Example: Filtering individual rows before grouping
SELECT Name, Salary
```

```
FROM Employees
WHERE Salary > 5000; [cite: 130-132]

-- HAVING Example: Filtering aggregated groups after grouping
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 5000; [cite: 134-137]
```

Input Data (Employees Table)

| Name | Department | Salary |

| :--- | :--- | :--- |

| Alice | HR | 4000 |

| Bob | HR | 4500 |

| Charlie | IT | 6000 |

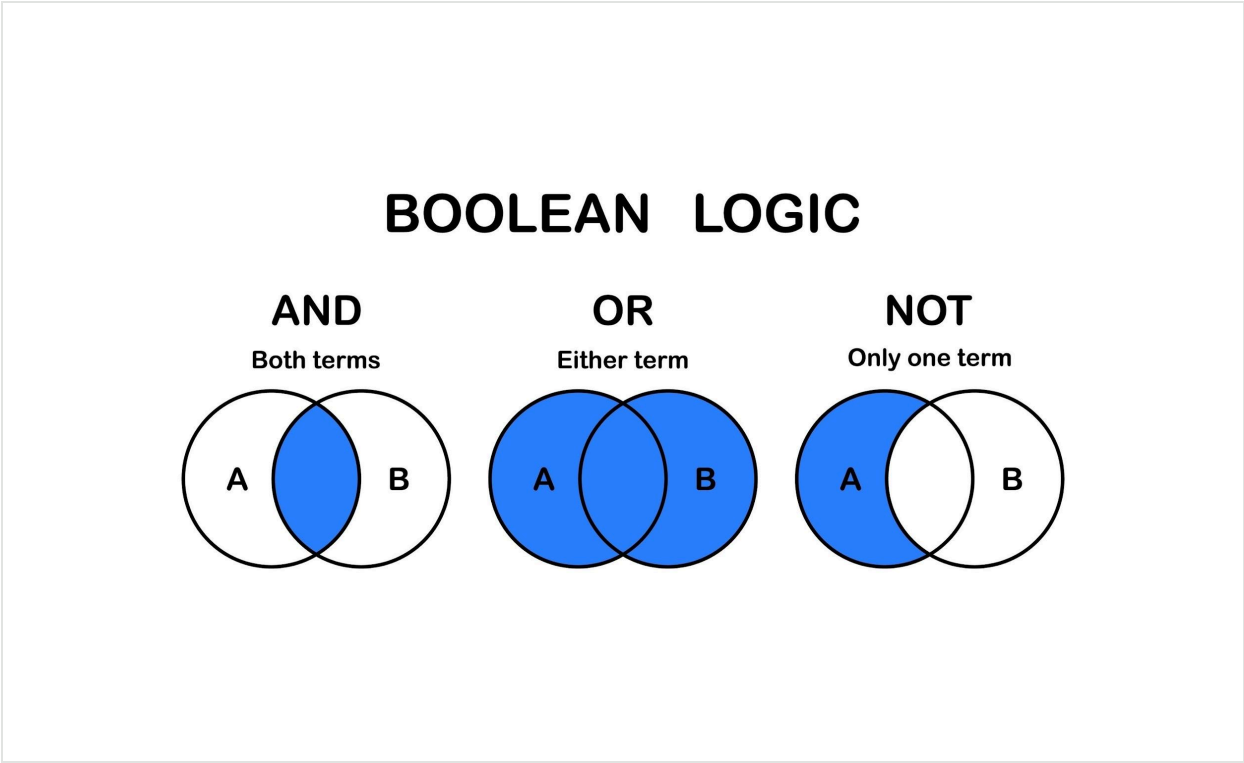
| Diana | IT | 7000 |

Output Table (For the HAVING Query)

First, the database groups the HR department (Average = 4250) and the IT department (Average = 6500). Then, the `HAVING` clause filters out the HR department because its average is not greater than 5000.

Department	AvgSalary
IT	6500

Concept 12: What are Joins in SQL?



Shutterstock

Joins are incredibly powerful tools used to combine rows of data from two or more tables based on a related, common column between them. In a relational database, data is deliberately split up to avoid redundancy (normalization); Joins are how you stitch that data back together for analysis.

Here are the main types of Joins :

1. **INNER JOIN:** Returns only matching rows from both tables.
2. **LEFT JOIN:** Returns all rows from the left table, plus matching rows from the right.
3. **RIGHT JOIN:** Returns all rows from the right table, plus matching rows from the left.
4. **FULL JOIN:** Returns all rows from both tables (matches and non-matches).
5. **SELF JOIN:** Joins a table with itself.
6. **CROSS JOIN:** Returns the Cartesian product (all possible combinations) of both tables.

Concept 13: What is INNER JOIN?

An `INNER JOIN` is the most commonly used join. It combines rows from two tables, but it *only* returns the records where the joining condition is explicitly met in **both** tables. It completely ignores unmatched rows.

The Query

```SQL
```

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
INNER JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID; [cite: 167-169]
```

Input Data (Before Query Execution)

Employees Table (Left Table)

| EmployeeID | Name | DepartmentID |

| :--- | :--- | :--- |

| 1 | Alice | 101 |

| 2 | Bob | 102 |

| 3 | Charlie | 999 (No matching Dept) |

Departments Table (Right Table)

| DepartmentID | DepartmentName |

| :--- | :--- |

| 101 | HR |

| 102 | IT |

| 103 | Marketing (No matching Emp) |

Output Table Charlie and the Marketing department are completely excluded because they do not have a match in the opposite table.

Name	DepartmentName
Alice	HR
Bob	IT

Concept 14: What is LEFT JOIN?

A LEFT JOIN guarantees that you will get **all** the records from the "Left" table (the one mentioned first in the query), regardless of whether they have a match in the "Right" table.

+1

- If a match is found in the Right table, it brings that data in.
- +1
- If no match is found, the database simply fills the columns from the Right table with NULL values.
- +1

The Query

```SQL
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
LEFT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID; [cite: 186-188]
```

Input Data

(Using the exact same Input Tables from Concept 13)

Output Table Notice that Charlie is included because he is in the Left table (Employees), but his department is NULL because "999" doesn't exist in the Departments table. Marketing is excluded because it's only in the Right table.

Name	DepartmentName
Alice	HR
Bob	IT
Charlie	NULL

Concept 15: What is RIGHT JOIN?

A RIGHT JOIN is the exact opposite of a Left Join. It guarantees that you will get **all** the records from the "Right" table (the one mentioned second in the query).

+1

- If a match is found in the Left table, it brings that data in.

- If no match is found, the result will contain `NULL` values for the columns originating from the Left table.
- `+1`

The Query

```SQL
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
RIGHT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID; [cite: 202-209]
```

Input Data

(Using the exact same Input Tables from Concept 13)

Output Table Notice that the Marketing department is included because it is in the Right table (`Departments`), but the Employee Name is `NULL` because no employee is assigned to Dept 103. Charlie is completely excluded.

Name	DepartmentName
Alice	HR
Bob	IT
NULL	Marketing

Concept 16: What is FULL JOIN?

A `FULL JOIN` (or Full Outer Join) is essentially a combination of both a `LEFT JOIN` and a `RIGHT JOIN`. It returns absolutely all records from both tables, matching them up where a relationship exists.

`+1`

- It returns all rows from both the left and right tables.
- If there is no match for a particular row on either side, the missing side will just be filled with `NULL` values.
- `+1`

- It is extremely useful when you want to find unmatched records in both tables simultaneously.

The Query

```SQL
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
FULL JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID; [cite: 226-230]
```

Input Data (Before Query Execution)

Employees Table (Left Table)

| EmployeeID | Name | DepartmentID |

| :--- | :--- | :--- |

| 1 | Alice | 101 |

| 2 | Bob | 102 |

| 3 | Charlie | 999 (No matching Dept) |

Departments Table (Right Table)

| DepartmentID | DepartmentName |

| :--- | :--- |

| 101 | HR |

| 102 | IT |

| 103 | Marketing (No matching Emp) |

Output Table

Notice how it grabs everyone and every department. Charlie is there with a `NULL` department, and Marketing is there with a `NULL` employee.

Name	DepartmentName

Alice	HR
Bob	IT
Charlie	NULL
NULL	Marketing

Concept 17: What is Self Join?

A `SELF JOIN` is exactly what it sounds like: a regular join, but the table is joined with itself. This is used when you need to compare rows within the exact same table.

`+4`

- Because you are referencing the same table twice in one query, you **must** use table aliases (like `E1` and `E2`) to tell the database which "copy" of the table you are referring to.
- It is very commonly used for hierarchical data, like finding out who manages whom in an employee table.

The Query

```SQL
SELECT E1.EmployeeName AS Employee, E2.EmployeeName AS Manager
FROM Employees E1
JOIN Employees E2 ON E1.ManagerID = E2.EmployeeID; [cite: 243-246]
```

Input Data (Employees Table)

Notice how the `ManagerID` points back to the `EmployeeID` in the *same* table.

| EmployeeID | EmployeeName | ManagerID |

| :--- | :--- | :--- |

| 1 | Alice (The Boss) | NULL |

| 2 | Bob | 1 |

| 3 | Charlie | 1 |

Output Table

The query successfully matches Bob and Charlie's `ManagerID` (1) to Alice's `EmployeeID` (1).

Employee	Manager
Bob	Alice
Charlie	Alice

Concept 18: What is Cross Join?

A `CROSS JOIN` returns what is mathematically called the "Cartesian product" of two tables. This means it combines *every single row* from the first table with *every single row* from the second table.

`+1`

- It does not require any matching condition (no `ON` clause).
- The number of rows in the final output is calculated by multiplying the number of rows in Table 1 by the number of rows in Table 2.
- **Warning:** If you cross join two large tables, the result will be massive and could crash your query.

The Query

```SQL
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
CROSS JOIN Departments; [cite: 259-262]
```

Input Data

Employees Table (2 rows)

| Name |

| :--- |

| Alice |

| Bob |

Departments Table (3 rows)

| DepartmentName |

| :--- |

| HR |

| IT |

| Sales |

Output Table

(2 Employees * 3 Departments = 6 total rows)

Name	DepartmentName
Alice	HR
Alice	IT
Alice	Sales
Bob	HR
Bob	IT
Bob	Sales

Concept 19: What is Union and Union All?

While Joins combine tables horizontally (adding columns side-by-side), `UNION` and `UNION ALL` combine the result sets of two or more `SELECT` statements vertically (stacking rows on top of each other) .

The Queries

```SQL
-- UNION Example (Removes Duplicates)
SELECT Name FROM Employees
UNION
SELECT Name FROM Managers; [cite: 272-275]

-- UNION ALL Example (Keeps Duplicates)
SELECT Name FROM Employees
UNION ALL
SELECT Name FROM Managers; [cite: 277-280]
```

Input Data (Before Query Execution)

Employees Table

| Name |

| :--- |

| Alice |

| Bob |

Managers Table

| Name |

| :--- |

| Bob |

| Charlie |

Output Table (UNION) Notice "Bob" is only listed once because `UNION` removes duplicates. | Name | | :--- | | Alice | | Bob | | Charlie |

Output Table (UNION ALL) Notice "Bob" is listed twice. `UNION ALL` just stacks them blindly without filtering. | Name | | :--- | | Alice | | Bob | | Bob | | Charlie |

Concept 20: Difference between UNION and UNION ALL

This is a very common interview question. Here is a quick reference table summarizing their differences .

Criteria	UNION	UNION ALL

Duplicates	Removes duplicate rows.	Includes all duplicate rows.
Performance	Slower (due to the extra work of duplicate removal).	Faster (no duplicate removal step).
Sorting	Automatically sorts the result set.	Does not sort the result set.
Usage	Used when duplicate data is not required.	Used when duplicate data needs to be preserved.

Concept 21: What is Normalization?

Normalization is the systematic process of organizing data within a database to minimize redundancy (duplicate data) and improve overall data integrity. Think of it as decluttering your database. It involves taking large, clunky tables and dividing them into smaller, logically related tables, then defining the relationships between them using Foreign Keys.

+1

Key Benefits:

- Reduces data redundancy (saving storage space).
- Improves data consistency (if you need to update a department name, you only do it in one place).
- Simplifies data maintenance and increases integrity.

The Normal Forms (Stages of Normalization):

1. **1NF (First Normal Form):** Eliminates duplicate columns and ensures every column contains atomic (indivisible) values. No lists inside a single cell!
2. **2NF (Second Normal Form):** Must be in 1NF. Ensures no partial dependency; all non-key attributes must depend on the *entire* primary key.
3. **3NF (Third Normal Form):** Must be in 2NF. Removes transitive dependencies (where a non-key column depends on another non-key column).
4. **BCNF (Boyce-Codd Normal Form):** A stricter version of 3NF ensuring every determinant is a candidate key.

Input Data (Unnormalized Table) Notice how "IT" and "Bangalore" are repeated. If IT moves to Mumbai, you have to update multiple rows. | EmployeeID | EmployeeName | Department | DepartmentLocation | | :--- | :--- | :--- | :--- | | 101 | Vinay | IT | Bangalore | | 102 | Awadhesh | IT | Bangalore |

Output Data (Normalized Tables - 1NF & 2NF) We split them into two tables. Now, "IT" is only stored once.

+1
Employee Table | EmployeeID | EmployeeName | DepartmentID | | :--- | :--- | :--- | | 101 | Vinay | 1 | | 102 | Awadhesh | 1 |

Department Table | DepartmentID | DepartmentName | Location | | :--- | :--- | :--- | | 1 | IT | Bangalore |

Concept 22: What is Denormalization?

Denormalization is the exact opposite of Normalization. It is the deliberate process of combining tables or adding redundant data back into a database.

Why would you do this? **Speed.** Normalization requires complex JOIN operations to stitch data back together, which can be slow. Denormalization is used when fast data retrieval (reading data) is vastly more important than saving storage space or strictly maintaining data integrity (like in data warehouses or reporting systems).

+1
Key Features:

- Improves data retrieval speed significantly.
- Reduces the number of JOINs required in your queries.
- Increases data redundancy (you will have duplicate data).

Input Data (Normalized Tables)

Employee Table: 101 (Vinay, Dept 1), 102 (Awadhesh, Dept 2). *Department Table:* 1 (IT), 2 (HR).

+1
Output Data (Denormalized Table) Notice how we brought the DepartmentName directly into the Employee table. It's faster to read, but redundant.

+1

EmployeeID	EmployeeName	DepartmentName
101	Vinay	IT
102	Awadhesh	HR

Concept 23: What is the Difference Between CHAR and VARCHAR?

Both CHAR and VARCHAR store string data (text), but they allocate memory completely differently.

Criteria	CHAR (Character)	VARCHAR (Variable Character)
Storage & Memory	**Fixed-length.** It always uses the specified length in memory.	**Variable-length.** It only uses the space required for the actual data, plus 1-2 bytes to record the length.
Padding	Pads extra spaces to the right to match the fixed length.	Does not pad spaces.
Performance	Faster for fixed-size data.	Slower for varying data sizes (due to calculating length).
Best Use Case	When data length is strictly consistent (e.g., a 2-letter state code like 'NY', or an 8-character PIN).	When data length varies wildly (e.g., names, emails, addresses).

The Query

```SQL
CREATE TABLE Employees (
   EmpID INT,
   Name CHAR(10),
   Address VARCHAR(50)
); [cite: 331-335]
```

Under the Hood Detail: If you insert the name "Bob" into the Name CHAR(10) column, it will store "Bob " (padding it with 7 spaces to reach 10 bytes). If you insert "Bob" into Address VARCHAR(50), it simply stores "Bob" and takes up roughly 4 bytes.

Concept 24: What is the difference between SQL and MySQL?

This is a classic beginner trap. People often use these terms interchangeably, but they are fundamentally different things.

- **SQL (Structured Query Language):** This is the actual **language**. It is the standard syntax (developed by ANSI) used to write queries and communicate with databases. You cannot "install" SQL on your computer; it's just a language, like English.
- **MySQL:** This is a piece of **software**. Specifically, it is a Relational Database Management System (RDBMS) currently developed by Oracle. It is the actual engine that stores, manages, and retrieves the data.

Analogy: SQL is the English language. MySQL is a person who understands English and executes your commands. Other "people" who understand SQL include PostgreSQL, SQL Server, and Oracle DB.

Example:

- **SQL:** The query string itself: `SELECT * FROM Employees;`
- **MySQL:** The software that receives that string, searches its hard drive for the `Employees` table, and returns the data to your screen.

Concept 25: What is Auto Increment in SQL?

Auto Increment is a property applied to a column that automatically generates a unique, sequential number every time a new row is inserted into the table.

You don't have to manually figure out what the next ID should be; the database handles it for you. It is almost exclusively used for generating `Primary Key` values.

`+1`

- By default, it usually starts at `1` and increments by `1` for each new row.

The Query (MySQL Syntax)

```SQL
CREATE TABLE Employees (
   EmployeeID INT AUTO_INCREMENT PRIMARY KEY,
   Name VARCHAR(50),
   Salary DECIMAL(10, 2)
); [cite: 353-357]
```

Input Data (To Insert) Notice that we completely omit the `EmployeeID` in our insert statements.

```SQL
INSERT INTO Employees (Name, Salary) VALUES ('Vinay', 50000); [cite: 360]
INSERT INTO Employees (Name, Salary) VALUES ('Raj', 40000); [cite: 360]
```

Output Table The database automatically assigned `1` and `2` to the `EmployeeID` column.

EmployeeID	Name	Salary
1	Vinay	50000.00
2	Raj	40000.00

Concept 26: What is Subquery?

A Subquery is essentially a query nested inside another query in SQL. It is used to fetch preliminary data that the main (outer) query will then use as a condition to further filter or manipulate the final result.

+1

Key Characteristics:

- It is also referred to as an Inner Query or Nested Query.
- The subquery almost always executes *before* the main query.
- The result of the subquery is passed directly to the Outer Query.
- Subqueries can return a single row (one value), multiple rows, or be "correlated" (which we will cover in concept 28).
- +2

The Query (Single Row Subquery Example)

This query finds the employee(s) who make the absolute highest salary in the company.

```
SQL

SELECT name, salary
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees); [cite: 380-382]
```

Input Data (Employees Table)

| name | salary |

| :--- | :--- |

| Alice | 50000 |

| Bob | 80000 |

| Charlie | 60000 |

| Diana | 80000 |

Output Table

First, the inner query `(SELECT MAX(salary) FROM employees)` runs and returns `80000`. Then, the outer query runs as if it says `WHERE salary = 80000`.

| name | salary |

| :--- | :--- |

| Bob | 80000 |

| Diana | 80000 |

Concept 27: What is Nested Query?

A Nested Query is exactly the same concept as a Subquery; the terms are used interchangeably in the industry. It is a query written inside another query to retrieve data based on the result of the inner query. It helps break down complex logic into smaller, manageable steps.

+2

The Query

Let's find all employees who work in the 'Sales' department, assuming we only know the department name, not its ID.

```SQL
SELECT name
FROM employees
WHERE department_id = (SELECT department_id FROM departments WHERE department_name = 'Sales'); [cite: 408-410]
```

Input Data

Departments Table

| department_id | department_name |

| :--- | :--- |

| 1 | HR |

| 2 | Sales |

Employees Table

| name | department_id |

| :--- | :--- |

| Alice | 1 |

| Bob | 2 |

Output Table The inner query runs first, finding that 'Sales' has a `department_id` of 2. The outer query then fetches everyone with a `department_id` of 2 . | name | | :--- | | Bob |

Concept 28: What is Correlated Subquery?

A Correlated Subquery is a highly specific, more complex type of subquery. Unlike a regular subquery (which runs just once), a correlated subquery depends on values from the *outer* query to execute.

Because it relies on the outer query, the inner query must execute repeatedly—once for **every single row** evaluated by the outer query. This makes it much slower and more resource-intensive than a standard subquery.

`+2`

The Query

Let's find employees whose salary is higher than the average salary *of their specific department*.

```
SQL

SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE department_id =
e.department_id); [cite: 428-430]
```

Input Data (Employees Table)

| name | salary | department_id |

| :--- | :--- | :--- |

| Alice | 40000 | 1 (HR Avg: 45k) |

| Bob | 50000 | 1 (HR Avg: 45k) |

| Charlie | 60000 | 2 (IT Avg: 70k) |

| Diana | 80000 | 2 (IT Avg: 70k) |

Output Table For Alice, the inner query calculates the HR average (45k). Alice makes 40k, so she is filtered out. For Bob, it calculates HR average (45k). Bob makes 50k, so he is included. This row-by-row check continues for everyone . | name | salary | | :--- | :--- | | Bob | 50000 | | Diana | 80000 |

Concept 29: What is GROUP BY in SQL?

The GROUP BY clause is a powerful summarization tool. It is used to group rows that have the exact same values in specified columns into "summary rows".

It is almost exclusively used alongside **aggregate functions**, which perform a calculation on each newly formed group of data.

- COUNT(): Counts the number of rows in the group.
- SUM(): Calculates the total sum.
- AVG(): Calculates the average value.
- MAX() / MIN(): Returns the highest or lowest value in the group.

(Crucial syntax note: It must always be placed after the WHERE clause but before the ORDER BY clause).

The Query Let's find the total sales amount for each product category.

```
SQL

SELECT category, SUM(sales_amount) AS Total_Sales
FROM sales
GROUP BY category; [cite: 453-455]
```

Input Data (Sales Table)

| category | sales_amount |

| :--- | :--- |

| Electronics | 100 |

| Clothing | 50 |

| Electronics | 200 |

| Clothing | 75 |

Output Table

The database bunches the "Electronics" rows together and sums them (100+200), and does the same for "Clothing".

| category | Total_Sales |

| :--- | :--- |

| Electronics | 300 |

| Clothing | 125 |

Concept 30: Difference Between GROUP BY and ORDER BY

While both clauses are used to organize the results of your query, they do completely different things. `GROUP BY` physically squashes rows together to summarize them, while `ORDER BY` simply shuffles the display order of the final result .

Feature	GROUP BY	ORDER BY
Primary Function	Used to group rows based on the same values in one or more columns.	Used to sort the result set in ascending (ASC) or descending (DESC) order.
Aggregate Functions	Always works in tandem with aggregate functions like COUNT(), SUM(), AVG(), etc.	Does not require aggregate functions at all.
Syntax Order	Syntax: Comes *before* ORDER BY.	Syntax: Always comes *after* GROUP BY.
Result	Groups the result into summary rows (reduces total row count).	Sorts the entire result set (does not change total row count).

Concept 31: What is the Use of LIMIT in SQL?

The `LIMIT` clause is a very straightforward but essential tool used to restrict the maximum number of rows returned by a query. Instead of dumping thousands of records onto your screen, you can tell the database to just give you a specific number.

- It is commonly used to fetch "Top N" records (like the top 5 highest-paid employees).

- It is also widely used for Pagination on websites (e.g., showing only 10 results per page).

The Query Let's fetch the top 3 highest salaries.

SELECT name, salary

FROM employees

ORDER BY salary DESC

[cite_start]LIMIT 3; [cite: 482-485]

Input Data (Employees Table) | name | salary | | :--- | :--- | | Alice | 45000 | | Bob | 80000 | | Charlie | 60000 | | Diana | 90000 | | Evan | 50000 |

Output Table First, the database sorts them from highest to lowest (`ORDER BY salary DESC`). Then, `LIMIT 3` chops off everyone after the 3rd row. | name | salary | | :--- | :--- | | Diana | 90000 | | Bob | 80000 | | Charlie | 60000 |

Concept 32: How to Find the Second Highest Salary in SQL?

This is easily one of the most frequently asked SQL interview questions. While you *could* use `LIMIT` (with an `OFFSET`), the classic, foolproof way to do this across almost all relational databases is by using a Subquery.

The Query

SELECT MAX(salary)

FROM employees

[cite_start]WHERE salary < (SELECT MAX(salary) FROM employees); [cite: 489-491]

How it Works (Step-by-Step):

1. **The Inner Query:** `(SELECT MAX(salary) FROM employees)` runs first. It scans the whole table and finds the absolute highest salary. Let's say that is 90,000.

2. **The Outer Query:** The database substitutes that 90,000 into the outer query. It now reads: `WHERE salary < 90000`. It then searches the remaining salaries for the highest one that is strictly *less* than 90,000.

Input Data (Employees Table) | name | salary | | :--- | :--- | | Alice | 45000 | | Bob | 80000 | | Charlie | 60000 | | Diana | 90000 |

Output Table The highest below 90,000 is 80,000. | MAX(salary) | | :--- | | 80000 |

Concept 33: How to find Duplicate Records in a table?

To hunt down duplicate records, you combine the `GROUP BY` clause (to bunch identical records together) with the `HAVING` keyword (to filter out the groups that only have one record).

The Query

SELECT column_name, COUNT(*)

FROM table_name

GROUP BY column_name

[cite_start]HAVING COUNT(*) > 1; [cite: 497-500]

Input Data (Employees Table) | name | department | | :--- | :--- | | Alice | HR | | Bob | IT | | Charlie | IT | | Alice | Sales |

Output Table The query groups the names together. "Alice" appears twice, "Bob" once, and "Charlie" once. `HAVING COUNT(*) > 1` filters out Bob and Charlie, leaving only the duplicate name. | name | COUNT(*) | | :--- | :--- | | Alice | 2 |

Concept 34: What is CTE (Common Table Expression)?

A CTE (Common Table Expression) allows you to create a temporary, named result set that you can reference within a `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement.

Think of a CTE as a temporary "virtual table" that exists *only* for the duration of that single query. It is a fantastic alternative to subqueries because it makes complex SQL code much easier to read, organize, and debug.

The Query Let's first isolate IT employees into a CTE, and then query *that* CTE to find the ones making over 50,000.

WITH IT_Employees AS (

```
    SELECT name, department, salary

    FROM employees

    WHERE department = 'IT'

)

SELECT name, salary

FROM IT_Employees
```

[cite_start]WHERE salary > 50000; [cite: 515-522]

Explanation:

1. The code block inside the `WITH IT_Employees AS (...)` creates the temporary result set containing only IT staff.

2. The `SELECT` statement immediately following it treats `IT_Employees` like a real table and filters it for high salaries.

Concept 35: What is Temporary Table in SQL?

A Temporary Table is an actual table created in the database to temporarily store data, but it only exists during your current database session. As soon as you log off or close your connection to the database, the table is automatically deleted.

- They are used to store intermediate results when dealing with massive, complex multi-step processes.

- In MySQL, you use the keyword `TEMPORARY`. In SQL Server, you prefix the table name with a `#` (e.g., `#temp_table`).

The Query (MySQL) Let's create a temporary table to store high-earning employees.

```
CREATE TEMPORARY TABLE temp_emp AS

SELECT name, salary

FROM employees
```

WHERE salary > 50000;

[cite_start]SELECT * FROM temp_emp; [cite: 545-549]

Explanation: Unlike a CTE (which vanishes the moment the query finishes running), `temp_emp` will stick around. You can run `SELECT`, `UPDATE`, or `JOIN` queries against `temp_emp` as many times as you want until your session ends, at which point it vanishes into thin air .

Concept 36: What is Window Function in SQL?

A Window Function is an advanced SQL feature that performs calculations across a specific set of table rows that are related to the current row .

How it differs from `GROUP BY`: When you use aggregate functions with `GROUP BY`, it collapses the rows into a single summary row. A Window Function, on the other hand, calculates an aggregate value based on a group of rows (the "window") but **does not collapse the result**; it keeps the original rows intact and simply adds the calculated value as a new column.

- It almost always works with the `OVER()` clause, which defines the "window".
- Inside `OVER()`, you use `PARTITION BY` (to divide data into groups) and `ORDER BY` (to sort data within those groups).
- Commonly used for calculating running totals, moving averages, and rankings.

The Query (Calculating a Running Total)

SELECT name, department, salary,

SUM(salary) OVER (PARTITION BY department ORDER BY salary) AS Running_Total

[cite_start]FROM employees; [cite: 580-581]

Input Data (Employees Table) | name | department | salary | | :--- | :--- | :--- | | Alice | HR | 40000 | | Bob | HR | 50000 | | Charlie | IT | 60000 | | Diana | IT | 80000 |

Output Table Notice how the rows aren't squashed together. The `Running_Total` adds the salaries up row-by-row within each department. | name | department | salary | Running_Total | | :--- | :--- | :--- | :--- | | Alice | HR | 40000 | 40000 | | Bob | HR | 50000 | 90000 (40k + 50k) | | Charlie | IT | 60000 | 60000 | | Diana | IT | 80000 | 140000 (60k + 80k) |

Concept 37: Difference Between ROW_NUMBER(), RANK(), and DENSE_RANK()

These are the three primary ranking Window Functions. They all assign numbers to rows based on their order, but they handle "ties" (duplicate values) very differently .

Function	Purpose	Handles Duplicates	Skips Numbers?	Example Sequence
ROW_NUMBER()	Assigns a strictly unique, sequential number to every row.			

| Ignores ties; arbitrarily breaks them.

| No.

| 1, 2, 3, 4

| | **RANK()** | Assigns the exact same rank to tied rows, but skips the next numbers to make up for the tie.

| Gives identical ranks to ties.

| Yes.

| 1, 2, 2, 4

| | **DENSE_RANK()** | Assigns the same rank to tied rows, but does *not* skip the following numbers.

| Gives identical ranks to ties.

| No.

| 1, 2, 2, 3

|

The Query

```
SELECT name, score,
    ROW_NUMBER() OVER(ORDER BY score DESC) as row_num,
    RANK() OVER(ORDER BY score DESC) as rank_val,
    DENSE_RANK() OVER(ORDER BY score DESC) as dense_rank_val
FROM students;
```

Output Table (Demonstrating the differences) Look closely at how Bob and Charlie (who both scored 85) are handled by each function. | name | score | row_num | rank_val | dense_rank_val | | :--- | :--- | :--- | :--- | :--- | | Alice | 95 | 1 | 1 | 1 | | Bob | 85 | 2 | 2 | 2 | | Charlie | 85 | 3 | 2 | 2 | | Diana | 70 | 4 | 4 | 3 |

Concept 38: What is CASE Statement in SQL?

The `CASE` statement is SQL's version of `IF-THEN-ELSE` logic . It allows you to evaluate conditions and return specific values depending on which condition is met first. Once a condition is true, it stops reading and returns the result. If no conditions are true, it returns the value in the `ELSE` clause .

The Query Let's categorize employees based on their salary tiers.

```
SELECT name, salary,
    CASE
        WHEN salary > 6000 THEN 'High Salary'
        WHEN salary BETWEEN 4000 AND 6000 THEN 'Medium Salary'
        ELSE 'Low Salary'
    END AS Salary_Category
```

[cite_start]FROM employees; [cite: 615-621]

Input Data (Employees Table) | name | salary | | :--- | :--- | | Alice | 7500 | | Bob | 5000 | | Charlie | 3000 |

Output Table The database evaluates the `CASE` statement row by row, creating the new `Salary_Category` column dynamically. | name | salary | Salary_Category | | :--- | :--- | :--- | | Alice | 7500 | High Salary | | Bob | 5000 | Medium Salary | | Charlie | 3000 | Low Salary |

Concept 39: What is COALESCE in SQL?

`COALESCE` is an incredibly useful function that takes a list of columns or expressions and returns the **very first non-null value** it finds, reading from left to right.

- It is the standard way to handle `NULL` values or provide a fallback option .
- It can accept as many expressions/arguments as you want.

The Query Imagine you want to contact a student. You prefer email, but if they don't have an email, you'll take their phone number. If they have neither, you want it to explicitly say 'No Contact'.

SELECT name,

COALESCE(email, phone, 'No Contact') AS Contact_Info

[cite_start]FROM students; [cite: 627-628]

Input Data (Students Table) | name | email | phone | | :--- | :--- | :--- | | Alice | alice@email.com | 555-1234 | | Bob | NULL | 555-9876 | | Charlie | NULL | NULL |

Output Table | name | Contact_Info | | :--- | :--- | | Alice | alice@email.com | | Bob | 555-9876 | | Charlie | No Contact |

Concept 40: What is NVL Function in SQL?

The `NVL` function is very similar to `COALESCE`—it replaces `NULL` values with a specified default value.

However, there is a catch: `NVL` is specific to Oracle databases. Also, unlike `COALESCE` which can take an unlimited number of arguments, `NVL` strictly takes exactly two arguments. (Note: In MySQL, the equivalent is `IFNULL()`, and in SQL Server, it is `ISNULL()`).

The Query

SELECT name,

NVL(email, 'No Email') AS Email

[cite_start]FROM students; [cite: 637-638]

Output Table If the email column is NULL, it swaps it out with the string 'No Email'. | name | Email | | :--- | :--- | | Alice | alice@email.com | | Bob | No Email |

Here is the next batch of notes covering Concepts 41 through 45. This section introduces database performance optimization through indexing, as well as how to create virtual tables using Views.

Concept 41: What is Indexing in SQL?

Indexing is a powerful performance optimization technique used to vastly improve the speed of data retrieval (SELECT queries) from a database table .

Think of it like the index at the back of a textbook. Instead of reading every single page of the book to find a specific topic (a "Full Table Scan"), the database looks up the topic in the index to find the exact page number, and jumps straight there.

Key Features:

- Speeds up `SELECT` queries dramatically .
- It is automatically maintained by the database behind the scenes.
- You can create an index on one column or a combination of columns.
- **The Catch:** While it speeds up reading data, it slows down writing data (`INSERT`, `UPDATE`, `DELETE` operations) because the index itself must be updated every time the table changes.

The Query

```
SQL

CREATE INDEX idx_name ON students (name); [cite: 640-643]
```

Conceptual Effect (Input vs Output)

Indexing doesn't change the actual output of your queries; it changes *how fast* you get them.

If you have 10 million students and search for `WHERE name = 'Vinay'`, without an index, the database checks all 10 million rows. With the `idx_name` index, it instantly finds 'Vinay' in its sorted lookup tree and jumps to that specific row.

Concept 42: What is Clustered Index in SQL?

A Clustered Index is a specific type of index that determines the actual **physical sorting order** of the data on the hard drive .

Because the data itself can only be physically sorted in one specific order (e.g., sequentially from 1 to 100), a table can only have **one** clustered index .

- It is incredibly fast for data retrieval because the data is stored exactly how the index is structured.
- In almost all relational databases, when you create a Primary Key, the database automatically creates a Clustered Index on that column by default.

The Query

```SQL
CREATE CLUSTERED INDEX idx_id ON students (id); [cite: 652-655]
```

Input Data (Unsorted Disk Storage)

Imagine inserting records out of order.

| id | name |

| :--- | :--- |

| 3 | Charlie |

| 1 | Alice |

| 2 | Bob |

Output Table (Physical Storage after Clustered Index) The database physically rearranges the rows on the disk to match the clustered index order. | id | name | | :--- | :--- | | 1 | Alice | | 2 | Bob | | 3 | Charlie |

Concept 43: What is Non-Clustered Index in SQL?

A Non-Clustered Index creates a completely separate, secondary lookup structure that lives away from the actual table data .

Unlike a clustered index, it does not alter the physical order of the data in the table. Instead, it acts like a traditional book index, storing the sorted column values alongside a "pointer" (a specific address) that directs the database to the actual row .

- Because it doesn't rearrange the physical data, you can have **multiple** Non-Clustered Indexes on a single table .
- It improves search performance, but is slightly slower than a clustered index because the database has to do a two-step process: find the value in the index, then follow the pointer to the row.

The Query

```SQL
CREATE INDEX idx_name ON students (name); [cite: 662-665]
```

Conceptual Output

The table data stays wherever it was put. The database builds a hidden reference list that looks like this:

- Alice -> points to Row 2
- Bob -> points to Row 3
- Charlie -> points to Row 1

Concept 44: Difference between Clustered and Non-Clustered Index

This is a mandatory concept to know for performance tuning and interviews. Here is the breakdown:

+1

Feature	Clustered Index	Non-Clustered Index
Physical Storage	Sorts and stores data physically on the disk.	Does not affect physical order; stores pointers to data.
Quantity Limit	Only ONE allowed per table.	Multiple indexes allowed per table.
Speed	Faster for data retrieval.	Slower than Clustered Index (due to pointer lookup).

Creation	Automatically created on a Primary Key by default.	Must be manually created on specific columns.

Concept 45: What is View in SQL?

A View is essentially a "virtual table" that is generated dynamically based on the result of a predefined SQL query .

A View does not actually store any data physically on the hard drive (it just stores the query itself). Whenever you query the View, the database runs the underlying query against the real tables and presents the result as if it were a solid table .

- **Security:** It provides data security by allowing users to see specific columns or rows without giving them access to the entire underlying table.
- **Simplicity:** It simplifies complex queries. Instead of writing a massive 5-table `JOIN` every time, you write it once, save it as a View, and just `SELECT *` from the View.

The Query

Let's create a view that hides the HR details and only shows employees making good money.

```SQL
CREATE VIEW high_salary AS
SELECT name, salary
FROM employees
WHERE salary > 50000; [cite: 679-683]
```

Input Data (Real 'employees' Table)

| emp_id | name | salary | social_security_num |

| :--- | :--- | :--- | :--- |

| 1 | Alice | 45000 | 123-45 |

| 2 | Bob | 80000 | 987-65 |

| 3 | Charlie | 60000 | 555-55 |

Output Table (When you query `SELECT * FROM high_salary;`) Notice that Alice is filtered out, and the highly sensitive `social_security_num` column is completely hidden from anyone using this View. | name | salary | | :--- | :--- | | Bob | 80000 | | Charlie | 60000 |

Concept 46: Difference between View and Table

This is a great follow-up to understanding Views. While they might look identical when you run a `SELECT` statement, structurally they are very different .

Feature	View	Table
Structure	Virtual table.	Physical table.
Storage	Does not store data physically on the disk.	Stores data physically on the disk.
Foundation	Based strictly on SQL queries.	Contains the raw, actual data.
Security	Provides data security by restricting access to certain columns/rows.	No inherent data restriction unless specific permissions are applied.
Updates	Automatically updates when the base table data changes.	Needs manual updates (via `INSERT`, `UPDATE`, `DELETE`).

Concept 47: What is Stored Procedure?

A Stored Procedure is a group of predefined SQL statements that are saved directly inside the database so they can be executed multiple times .

Think of it like saving a complex script. Instead of writing out a 50-line query every time you want to run a monthly report, you write it once, wrap it in a Stored Procedure, and then just call the procedure by its name.

Key Points:

- It improves code reusability (write once, run anywhere).
- It increases performance because the database compiles and optimizes the execution plan the first time it is saved.

- It supports both input parameters (variables you pass in) and output parameters (results passed back out).
- It provides security by hiding the complex SQL logic from the end-user or the application.

The Query (Creating the Procedure)

```SQL
CREATE PROCEDURE GetEmployees
AS
BEGIN
   SELECT * FROM employees;
END; [cite: 702-706]
```

Executing the Procedure

To actually run it, you would use an execution command like this:

```SQL
EXEC GetEmployees;
```

Output Table

The output is simply the result of whatever SQL statements were inside the `BEGIN...END` block. In this case, the entire `employees` table.

| emp_id | name | salary |

| :--- | :--- | :--- |

| 1 | Alice | 45000 |

| 2 | Bob | 80000 |

Concept 48: Difference between Function and Stored Procedure

While both are blocks of saved code, they serve different purposes and have strict rules about how they can be used .

Feature	Function	Stored Procedure
Return Values	Always returns a value (either a single scalar value or a table).	May or may not return a value.
Usage in Queries	Can be used directly inside `SELECT`, `WHERE`, and `HAVING` statements.	Cannot be used inside `SELECT` statements (must be executed independently via `EXEC` or `CALL`).
Parameters	Allows only input parameters.	Allows both input and output parameters.
Database State	Cannot modify the database state (read-only).	Can modify the database state using `INSERT`, `UPDATE`, or `DELETE`.
Nesting	Cannot call Stored Procedures inside it.	Can easily call Functions inside it.
Primary Use Case	Used for complex calculations and returning specific values.	Used for executing heavy business logic and complex transactional operations.

Concept 49: What is Trigger in SQL?

A Trigger is an automated piece of SQL code that quietly waits in the background and is automatically executed (fired) whenever a specific event occurs in a table .

You **cannot** run a trigger manually. It is strictly bound to `INSERT`, `UPDATE`, or `DELETE` operations on a specific table. They are most commonly used to maintain strict audit logs or enforce complex data validation rules before bad data can enter the system.

`+3`

The Query

Let's create a trigger that automatically logs a message into a `logs` table every time a new employee is hired.

```
SQL

CREATE TRIGGER after_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
   INSERT INTO logs (message) VALUES ('New employee added');
END; [cite: 725-729]
```

Input Action (Triggering the Event)

A user runs this query on the `employees` table:

`INSERT INTO employees (name) VALUES ('Diana');`

Output Table (The `logs` Table)

Even though the user only interacted with the `employees` table, the database automatically ran the code inside the trigger behind the scenes.

| log_id | message | timestamp |

| :--- | :--- | :--- |

| 1 | New employee added | 2026-02-24 10:00:00 |

Concept 50: What is Cursor in SQL?

A Cursor is a database object used to retrieve, manipulate, and navigate through a result set **strictly row-by-row** .

Relational databases are designed for "set-based" operations (manipulating entire blocks of data simultaneously). Cursors break this rule by processing data sequentially. Because of this, they are notoriously slow and are **not recommended for large datasets** . They should only be used when complex data manipulation absolutely requires evaluating one specific row before moving to the next.

`+1`

The Query This is the standard life-cycle of a cursor: Declare it, Open it, Fetch the rows in a loop, Close it, and finally Deallocate it to free up server memory .

SQL

```sql
DECLARE emp_cursor CURSOR FOR
SELECT name FROM employees;

OPEN emp_cursor;

FETCH NEXT FROM emp_cursor INTO @emp_name;

WHILE @@FETCH_STATUS = 0
BEGIN
   PRINT @emp_name;
   FETCH NEXT FROM emp_cursor INTO @emp_name;
END;

CLOSE emp_cursor;
DEALLOCATE emp_cursor; [cite: 747-757]
```

How It Works (Conceptual Output)

Instead of outputting a grid/table all at once, the database loads the cursor into memory and prints the variables one at a time as the loop executes:

Alice

Bob

Charlie